TP 4, parties 3 et 4

3.A Préparer

Dataset

```
bq --location=EU mk --dataset $PROJECT:shopflow_rt
bq ls
```

Dataset (suite)

```
(tp4) mfpadonou@cloudshell:~/tp4 (project-64f95bfc-58b9-4be1-aa5)$ bq --locatio
n=EU mk --dataset $PROJECT:shopflow_rt
bq ls                                    # shopflow_rt doit apparaître
BigQuery error in mk operation: Dataset
'project-64f95bfc-58b9-4be1-aa5:shopflow_rt' already exists.
  datasetId
 -------------
  shopflow_rt
(tp4) mfpadonou@cloudshell:~/tp4 (project-64f95bfc-58b9-4be1-aa5)$
```

bq mk et bq ls

Droits du compte de service

```
(tp4) mfpadonou@cloudshell:~/tp4 (project-64f95bfc-58b9-4be1-aa5)$ SA=$(gcloud
projects describe $PROJECT --format='value(projectNumber)')-compute@developer.g
serviceaccount.com
for ROLE in roles/dataproc.worker roles/storage.objectAdmin roles/bigquery.data
Editor roles/bigquery.jobUser; do
  gcloud projects add-iam-policy-binding $PROJECT --member=serviceAccount:$SA -
-role=$ROLE --condition=None > /dev/null
done
gcloud projects get-iam-policy $PROJECT --flatten=bindings --filter="bindings.m
embers:$SA" --format='value(bindings.role)'
Updated IAM policy for project [project-64f95bfc-58b9-4be1-aa5].
Updated IAM policy for project [project-64f95bfc-58b9-4be1-aa5].
Updated IAM policy for project [project-64f95bfc-58b9-4be1-aa5].
Updated IAM policy for project [project-64f95bfc-58b9-4be1-aa5].
roles/bigquery.dataEditor
roles/bigquery.jobUser
roles/dataproc.worker
roles/storage.objectAdmin
(tp4) mfpadonou@cloudshell:~/tp4 (project-64f95bfc-58b9-4be1-aa5)$
```

Les 4 rôles du compte de service

Cluster

Commande de l'énoncé **sans les deux options d'arrêt automatique** :

```
gcloud dataproc clusters create shopflow-spark \
  --region=europe-west1 \
  --image-version=3.0-debian13 \
  --master-machine-type=n2-standard-4 --master-boot-disk-size=100GB \
  --num-workers=2 --worker-machine-type=n2-standard-4 --worker-boot-disk-size=100GB \
  --enable-component-gateway --public-ip-address \
  --delete-max-idle=30m --delete-max-age=4h
```

Cluster (suite)

```
(tp4) mfpadonou@cloudshell:~/tp4 (project-64f95bfc-58b9-4be1-aa5)$ time gcloud
dataproc clusters create shopflow-spark \
  --region=europe-west1 \
  --image-version=3.0-debian13 \
  --master-machine-type=n2-standard-4 --master-boot-disk-size=100GB \
  --num-workers=2 --worker-machine-type=n2-standard-4 --worker-boot-disk-size=1
00GB \
  --enable-component-gateway --public-ip-address
Waiting on operation [projects/project-64f95bfc-58b9-4be1-aa5/regions/europe-we
st1/operations/998d0e34-d7d7-3c47-b062-69b25828faec].
Waiting for cluster creation operation...
WARNING: For PD-Standard without local SSDs, we strongly recommend provisioning
 1TB or larger to ensure consistently high I/O performance. See https://cloud.g
oogle.com/compute/docs/disks/performance for information on disk I/O performanc
e.
WARNING: The firewall rules for specified network or subnetwork would allow ing
ress traffic from 0.0.0.0/0, which could be a security risk.
Waiting for cluster creation operation...done.
Created [https://dataproc.googleapis.com/v1/projects/project-64f95bfc-58b9-4be1
-aa5/regions/europe-west1/clusters/shopflow-spark] Cluster placed in zone [euro
pe-west1-b].

real    3m21.786s
user    0m1.310s
sys     0m0.136s
(tp4) mfpadonou@cloudshell:~/tp4 (project-64f95bfc-58b9-4be1-aa5)$
```

Création du cluster : 3 min 22 s

Cluster (suite)

```
(tp4) mfpadonou@cloudshell:~/tp4 (project-64f95bfc-58b9-4be1-aa5)$ gcloud datap
roc clusters describe shopflow-spark --region=europe-west1 \
  --format='yaml(status.state,config.softwareConfig.imageVersion,config.gceClus
terConfig.internalIpOnly,config.lifecycleConfig)'
config:
  gceClusterConfig:
    internalIpOnly: false
  softwareConfig:
    imageVersion: 3.0.3-debian13
status:
  state: RUNNING
(tp4) mfpadonou@cloudshell:~/tp4 (project-64f95bfc-58b9-4be1-aa5)$
```

describe : pas de lifecycleConfig

3.B Le job

Soumission

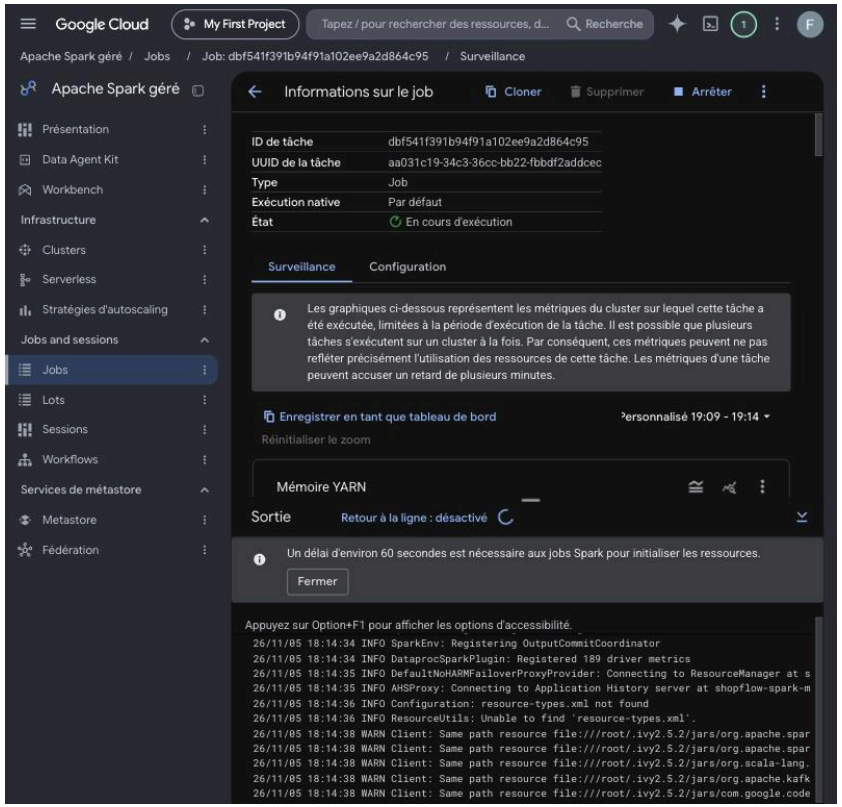
```
(tp4) mfpadonou@cloudshell:~/tp4 (project-64f95bfc-58b9-4be1-aa5)$ md5sum strea
m_agg.py; date -u +%T
gcloud dataproc jobs submit pyspark stream_agg.py \
  --cluster=shopflow-spark --region=europe-west1 \
  --properties=spark.jars.packages=org.apache.spark:spark-sql-kafka-0-10_2.13:4
.1.2 \
  -- $KAFKA_BOOTSTRAP $KAFKA_USER "$KAFKA_PASSWORD" $PROJECT-shopflow
0b6c27a24f86ed290165d22ae66bb245  stream_agg.py
18:14:08
Job [dbf541f391b94f91a102ee9a2d864c95] submitted.
Waiting for job output...
WARNING: Using incubator modules: jdk.incubator.vector
```

Soumission du job

Soumission (suite)

Le job `stream_agg.py` est soumis à **18:14:08 UTC**. La console montre le job : En cours d'exécution, dans la sortie, le téléchargement du connecteur Kafka depuis Maven Central (`spark.jars.packages`) :

Soumission (suite)



Le job dans la console

Soumission (suite)

Les trois requêtes démarrent à 18:14:50, 18:14:51 et 18:14:52.

Les premières lignes dans BigQuery

```
(tp4) mfpadonou@cloudshell:~/tp4 (project-64f95bfc-58b9-4be1-aa5)$ date -u +%T; bq query --use_legacy_sql
=false \
  'SELECT * FROM shopflow_rt.payments_by_minute ORDER BY window_start DESC LIMIT 20'
18:19:50
+---------------------+---------------------+--------------+------------+------------+------------+
|    window_start     |     window_end      | card_country | n_payments | n_declined | amount_sum |
+---------------------+---------------------+--------------+------------+------------+------------+
| 2026-11-05 18:15:00 | 2026-11-05 18:16:00 | BE           |         19 |          0 |    6680.82 |
| 2026-11-05 18:15:00 | 2026-11-05 18:16:00 | PT           |          5 |          0 |    3163.79 |
| 2026-11-05 18:15:00 | 2026-11-05 18:16:00 | GB           |         13 |          0 |    2578.43 |
| 2026-11-05 18:15:00 | 2026-11-05 18:16:00 | IT           |         28 |          0 |    5034.64 |
| 2026-11-05 18:15:00 | 2026-11-05 18:16:00 | NL           |         11 |          0 |    1642.24 |
| 2026-11-05 18:15:00 | 2026-11-05 18:16:00 | DE           |         36 |          1 |   12539.83 |
| 2026-11-05 18:15:00 | 2026-11-05 18:16:00 | CH           |          1 |          0 |      31.76 |
| 2026-11-05 18:15:00 | 2026-11-05 18:16:00 | ES           |         25 |          2 |    5899.41 |
| 2026-11-05 18:15:00 | 2026-11-05 18:16:00 | FR           |         91 |          2 |   22794.53 |
| 2026-11-05 18:15:00 | 2026-11-05 18:16:00 | US           |          8 |          1 |    3926.17 |
| 2026-11-05 18:14:00 | 2026-11-05 18:15:00 | GB           |          2 |          0 |     596.55 |
| 2026-11-05 18:14:00 | 2026-11-05 18:15:00 | CH           |          2 |          0 |     127.06 |
| 2026-11-05 18:14:00 | 2026-11-05 18:15:00 | US           |          1 |          0 |      31.45 |
| 2026-11-05 18:14:00 | 2026-11-05 18:15:00 | FR           |         14 |          1 |    1844.95 |
| 2026-11-05 18:14:00 | 2026-11-05 18:15:00 | ES           |          5 |          0 |    6579.55 |
| 2026-11-05 18:14:00 | 2026-11-05 18:15:00 | PT           |          1 |          0 |     133.86 |
| 2026-11-05 18:14:00 | 2026-11-05 18:15:00 | NL           |          3 |          0 |     354.55 |
| 2026-11-05 18:14:00 | 2026-11-05 18:15:00 | IT           |          1 |          0 |      74.51 |
| 2026-11-05 18:14:00 | 2026-11-05 18:15:00 | DE           |          5 |          0 |    2253.55 |
| 2026-11-05 18:14:00 | 2026-11-05 18:15:00 | BE           |          3 |          0 |     1270.3 |
+---------------------+---------------------+--------------+------------+------------+------------+
(tp4) mfpadonou@cloudshell:~/tp4 (project-64f95bfc-58b9-4be1-aa5)$
```

payments_by_minute à 18:19:50

Les premières lignes dans BigQuery (suite)

À 18:19, les fenêtres 18:14-18:15 et 18:15-18:16 du nouveau job sont en tête.

Mesure du délai (`lag`)

Requête de l'énoncé, une fois par minute :

Mesure du délai (`lag`) (suite)

```
(tp4) mfpadonou@cloudshell:~/tp4 (project-64f95bfc-58b9-4be1-aa5)$ for i in 1 2
 3 4; do bq query --use_legacy_sql=false \
  'SELECT CURRENT_TIMESTAMP() AS maintenant, MAX(window_end) AS derniere_fenetr
e,
         TIMESTAMP_DIFF(CURRENT_TIMESTAMP(), MAX(window_end), SECOND) AS retar
d_s
   FROM shopflow_rt.payments_by_minute'; sleep 60; done
+---------------------+---------------------+----------+
|     maintenant      |  derniere_fenetre   | retard_s |
+---------------------+---------------------+----------+
| 2026-11-05 18:26:02 | 2026-11-05 18:22:00 |      242 |
+---------------------+---------------------+----------+
+---------------------+---------------------+----------+
|     maintenant      |  derniere_fenetre   | retard_s |
+---------------------+---------------------+----------+
| 2026-11-05 18:27:05 | 2026-11-05 18:23:00 |      245 |
+---------------------+---------------------+----------+
+---------------------+---------------------+----------+
|     maintenant      |  derniere_fenetre   | retard_s |
+---------------------+---------------------+----------+
| 2026-11-05 18:28:08 | 2026-11-05 18:25:00 |      188 |
+---------------------+---------------------+----------+
+---------------------+---------------------+----------+
|     maintenant      |  derniere_fenetre   | retard_s |
+---------------------+---------------------+----------+
| 2026-11-05 18:29:13 | 2026-11-05 18:26:00 |      193 |
+---------------------+---------------------+----------+
(tp4) mfpadonou@cloudshell:~/tp4 (project-64f95bfc-58b9-4be1-aa5)$
```

lag, 4 mesures

Mesure du délai (`lag`) (suite)

maintenant	dernière fenêtre (`window_end`)	`lag`
18:26:02	18:22:00	242
18:27:05	18:23:00	245
18:28:08	18:25:00	**188**
18:29:13	18:26:00	193

Même requête toutes les ~11 s : la courbe est en **dents de scie**. `lag` augmente de 11 s à chaque mesure, puis retombe d'environ 50 s quand une nouvelle fenêtre arrive.

Mesure du délai (`lag`) (suite)

```
(tp4) mfpadonou@cloudshell:~/tp4 (project-64f95bfc-58b9-4be1-aa5)$ column -t -s
, retard_10s.log | head -30
maintenant  derniere_fenetre  retard_s
18:20:31    18:17:00          211
18:20:42    18:17:00          222
18:20:53    18:17:00          233
18:21:04    18:17:00          244
18:21:15    18:18:00          195
18:21:26    18:18:00          206
18:21:37    18:18:00          217
18:21:48    18:18:00          228
18:22:00    18:18:00          240
18:22:11    18:19:00          191
18:22:22    18:19:00          202
18:22:33    18:19:00          213
18:22:44    18:19:00          224
(tp4) mfpadonou@cloudshell:~/tp4 (project-64f95bfc-58b9-4be1-aa5)$
```

Dents de scie

Mesure du délai (`lag`) (suite)

Le **journal des chargements BigQuery** (`INFORMATION_SCHEMA.JOBS_BY_PROJECT`, `job_type = "LOAD"`) donne l'instant exact de chaque écriture :

Mesure du délai (`lag`) (suite)

```
(tp4) mfpadonou@cloudshell:~/tp4 (project-64f95bfc-58b9-4be1-aa5)$ bq query --u
se_legacy_sql=false --max_rows=30 \
  'SELECT FORMAT_TIMESTAMP("%T", creation_time) AS debut, FORMAT_TIMESTAMP("%T"
, end_time) AS fin,
         destination_table.table_id AS table_bq, total_bytes_processed AS octe
ts
   FROM `region-eu`.INFORMATION_SCHEMA.JOBS_BY_PROJECT
   WHERE job_type = "LOAD" AND creation_time BETWEEN TIMESTAMP "2026-11-05 18:1
7:00" AND TIMESTAMP "2026-11-05 18:20:20"
   ORDER BY creation_time'
+----------+----------+--------------------+--------+
|  debut   |   fin    |      table_bq      | octets |
+----------+----------+--------------------+--------+
| 18:17:03 | 18:17:06 | clicks_by_minute   |    103 |
| 18:17:04 | 18:17:07 | ip_alerts          |      0 |
| 18:17:05 | 18:17:08 | payments_by_minute |      0 |
| 18:17:33 | 18:17:35 | clicks_by_minute   |      0 |
| 18:17:34 | 18:17:37 | payments_by_minute |      0 |
| 18:17:35 | 18:17:38 | ip_alerts          |      0 |
| 18:18:03 | 18:18:06 | clicks_by_minute   |    103 |
| 18:18:04 | 18:18:07 | payments_by_minute |    440 |
| 18:18:05 | 18:18:07 | ip_alerts          |      0 |
| 18:18:33 | 18:18:36 | clicks_by_minute   |      0 |
| 18:18:34 | 18:18:36 | ip_alerts          |      0 |
| 18:18:35 | 18:18:37 | payments_by_minute |      0 |
| 18:19:03 | 18:19:05 | clicks_by_minute   |    103 |
| 18:19:04 | 18:19:06 | payments_by_minute |    440 |
| 18:19:05 | 18:19:06 | ip_alerts          |      0 |
| 18:19:34 | 18:19:36 | payments_by_minute |      0 |
| 18:19:34 | 18:19:38 | clicks_by_minute   |      0 |
| 18:19:35 | 18:19:38 | ip_alerts          |      0 |
| 18:20:03 | 18:20:08 | clicks_by_minute   |    103 |
| 18:20:04 | 18:20:07 | ip_alerts          |      0 |
| 18:20:05 | 18:20:07 | payments_by_minute |    440 |
+----------+----------+--------------------+--------+
(tp4) mfpadonou@cloudshell:~/tp4 (project-64f95bfc-58b9-4be1-aa5)$
```

Chargements BigQuery, un par micro-lot et par table

Mesure du délai (`lag`) (suite)

- La fenêtre 18:14-18:15 est chargée à **18:18:04 → 18:18:07**, et la fenêtre 18:15-18:16 à 18:19:04 → 18:19:06. Une fenêtre est donc écrite **3 min 07 s après sa fin**.
- Le connecteur lance **un job LOAD par micro-batch et par table, toutes les 30 s**, y compris quand il n'y a rien à écrire (`octets = 0`).

L'interface Spark

Onglet **Structured Streaming**

Défaut constaté : la page de détail d'une requête (*Streaming Query Statistics*) n'affiche aucun graphique.

test de `countDistinct`

job de test soumis au même cluster :

test de `countDistinct` (suite)

```
cat test_distinct.py
gcloud dataproc jobs submit pyspark test_distinct.py --cluster=shopflow-spark \

  --region=europe-west1 2>&1 | grep -E "RESULTAT|finished"
from pyspark.sql import SparkSession, functions as F
spark = SparkSession.builder.appName("test-distinct").getOrCreate()
s = (spark.readStream.format("rate").load()
     .withWatermark("timestamp", "2 minutes")
     .groupBy(F.window("timestamp", "1 minute"))
     .agg(F.countDistinct("value").alias("n")))
try:
    s.writeStream.format("console").outputMode("append").start()
except Exception as e:
    print("RESULTAT :", type(e).__name__, "-", str(e).splitlines()[0])
RESULTAT : AnalysisException - Distinct aggregations are not supported on strea
ming DataFrames/Datasets. Consider using approx_count_distinct() instead.;
Job [5309ca86bfc845a59b2298ce8206c4da] finished successfully.
(tp4) mfpadonou@cloudshell:~/tp4 (project-64f95bfc-58b9-4be1-aa5)$
```

countDistinct refusé

Réponses Q7 à Q9

Q7. L'événement de 10 h 00 m 30 s tombe dans la fenêtre [10:00, 10:01). Sa ligne apparaît dans BigQuery vers **10 h 04 m 07 s**, soit environ **3 min 37 s (env 3.5 min) après l'événement**. (la fenêtre 18:14-18:15 est chargée à 18:18:04-18:18:07).

Étape	Durée	Horloge
Attendre la fin de la fenêtre	30 s	10:01:00
Watermark de 2 min : la fenêtre n'est finale que lorsque Spark a vu un événement ≥ 10:03:00	120 s	10:03:00
Ces événements sont lus par le micro-lot **suivant** (déclencheur de 30 s) ; le watermark est recalculé **à la fin** de ce lot	30 s	10:03:30
Le nouveau watermark ne sert qu'au lot **d'après**, qui émet la fenêtre	30 s	10:04:00
Écriture indirecte dans BigQuery (Parquet sur GCS puis job LOAD)	~7 s	**10:04:07**

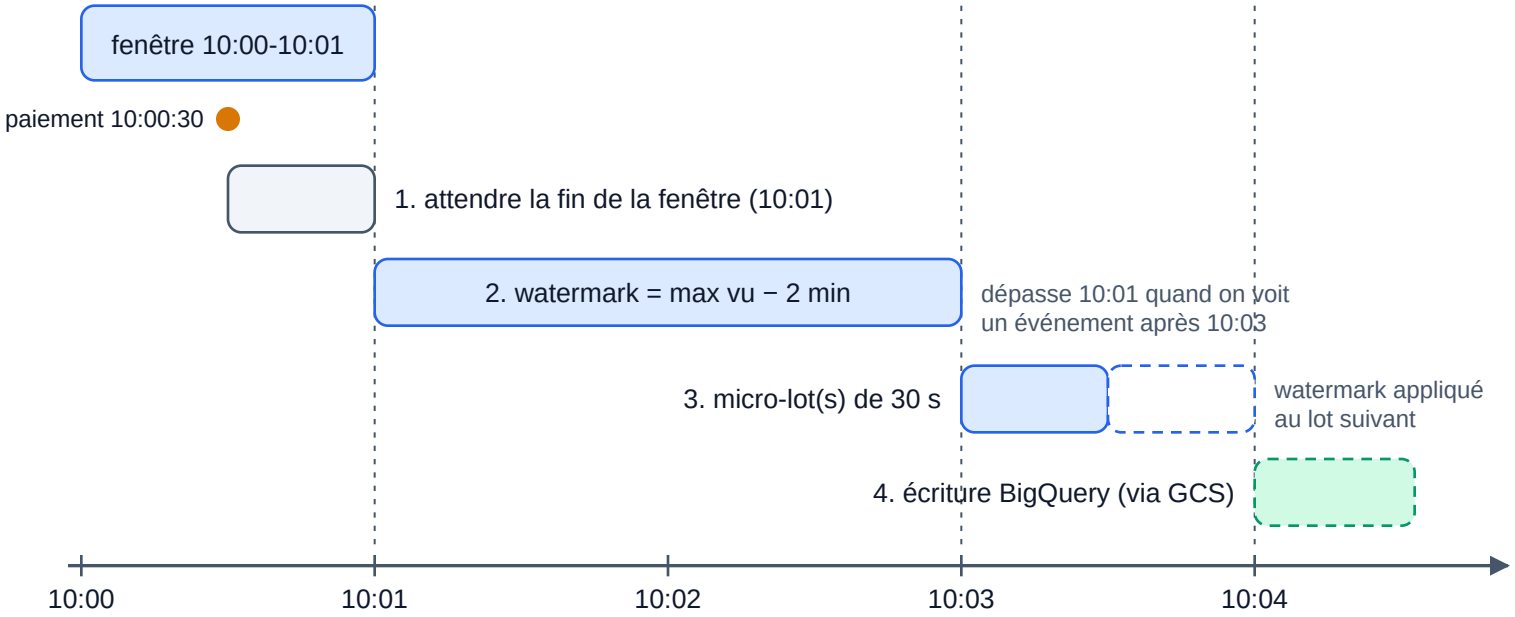
L'*écart du watermark* mesuré, ~150 s, confirme ce calcul : au début de chaque lot, le watermark vaut l'heure moins 2 min de seuil, moins les ~30 s qui séparent deux lots.

Réponses Q7 à Q9 (suite)

Le `**lag` mesuré oscille entre ~188 s et ~245 s**. Juste après l'écriture d'une fenêtre, il vaut 180 + 7 ≈ 188 s. Il augmente ensuite jusqu'à l'arrivée de la fenêtre suivante, 60 s plus tard.

Réponses Q7 à Q9 (suite)

Q7 : quand la fenêtre 10:00-10:01 arrive-t-elle ?



Ligne émise vers 10:03:30 à 10:04, soit 3 à 3,5 min après le paiement, + écriture BigQuery

Schéma : le délai s'additionne : fin de la fenêtre, 2 minutes de watermark, puis un ou deux micro-lots de 30 s avant l'écriture BigQuery (flux supposé sans retard, temps d'événement proche du temps de traitement).

Réponses Q7 à Q9 (suite)

Q8. `countDistinct` est **refusé à la création de la requête** : `AnalysisException - Distinct aggregations are not supported on streaming DataFrames/Datasets. Consider using approx_count_distinct() instead.` (capture ci-dessus).

Un comptage exact obligerait Spark à garder dans l'état **toutes les valeurs distinctes** de chaque groupe et de chaque fenêtre. `approx_count_distinct` est une approximation. Pour de petites valeurs, l'approximation reste exacte (`n_cards = n_tx = 17` pour 17 cartes réelles, voir Q10).

Q9. Valeurs pour `payments` (tableau de 3.B) :

Réponses Q7 à Q9 (suite)

- **débit d'entrée** moyen ≈ **3,7 msg/s**, très variable (0,5 à 8,2 msg/s). Les courbes des paiements et des clics montent et descendent **ensemble**, avec un cycle d'environ 13 min : le rejeu suit la densité du trafic d'origine ;
- **micro-lots** d'environ **9 à 10 s**, sauf le premier (27 à 33 s : démarrage et planification). L'essentiel est l'écriture BigQuery (`addBatch`) ;
- **lignes ignorées** : 29 sur 4 024 (**0,72 %**), et 0,97 % lors du premier essai. Sur `clicks`, 0,22 %.

Réponses Q7 à Q9 (suite)

Ces lignes ignorées sont les **messages publiés en retard** par le rejeu (environ 2 % du flux, d'après l'énoncé). Quand l'un d'eux arrive, son `event_ts` est déjà plus ancien que le watermark (temps d'événement maximal vu − 2 min). Sa fenêtre est fermée et écrite : Spark l'écarte, et l'agrégat ne sera **jamais corrigé**. Seuls les retards supérieurs à ~2 min 30 s (l'écart du watermark) sont perdus, ce qui explique que la mesure soit d'environ la moitié des 2 % de retardataires. Kafka, lui, n'a rien perdu : le message est dans le topic et dans le bronze. C'est le compromis du watermark : un seuil plus long rattrape plus de retardataires, mais retarde chaque fenêtre d'autant (Q7).

Deux remarques pour la séance :

Réponses Q7 à Q9 (suite)

- `by_country` et `ip_alerts` donnent exactement les mêmes chiffres d'entrée : **chaque requête lit `payments` dans Kafka de son côté**. Trois requêtes signifient trois lecteurs Kafka (groupes `spark-kafka-source-…`).
- Le tout premier lot accepte les retardataires (fenêtre 18:12 vue plus haut), puisque le watermark n'existe pas encore.

3.C Fraude visible ?

Pendant les 25 premières minutes du job, aucune IP n'a franchi le seuil de 3 cartes. Le générateur injecte les tests de cartes de manière irrégulière : une attaque en 27 min lors du premier essai, cinq en 80 min dans les lignes de la veille. **Prévoir que certains binômes devront attendre**, comme l'indique l'énoncé (« Si la table est vide, attendez quelques minutes »).

Une attaque est apparue à 18:42. Première requête de l'énoncé, lancée à 18:52 :

3.C Fraude visible ? (suite)

```
(tp4) mfpadonou@cloudshell:~/tp4 (project-64f95bfc-58b9-4be1-aa5)$ bq query --use_legacy_sql=false \
  'SELECT window_start, ip, n_cards, n_tx, n_declined
   FROM shopflow_rt.ip_alerts
   ORDER BY window_start DESC, n_cards DESC
   LIMIT 50'
+---------------------+-----------------+---------+------+------------+
|    window_start     |       ip        | n_cards | n_tx | n_declined |
+---------------------+-----------------+---------+------+------------+
| 2026-11-05 18:39:00 | 144.161.201.127 |      11 |   11 |         10 |
| 2026-11-05 18:38:00 | 144.161.201.127 |      15 |   15 |         13 |
| 2026-11-05 18:37:00 | 144.161.201.127 |      15 |   15 |         13 |
| 2026-11-05 18:36:00 | 144.161.201.127 |      15 |   15 |         13 |
| 2026-11-05 18:35:00 | 144.161.201.127 |      15 |   15 |         13 |
| 2026-11-05 18:34:00 | 144.161.201.127 |      15 |   15 |         13 |
| 2026-11-05 18:33:00 | 144.161.201.127 |      15 |   15 |         13 |
| 2026-11-05 18:32:00 | 144.161.201.127 |      15 |   15 |         13 |
| 2026-11-05 18:31:00 | 144.161.201.127 |      15 |   15 |         13 |
| 2026-11-05 18:30:00 | 144.161.201.127 |      15 |   15 |         13 |
| 2026-11-05 18:29:00 | 144.161.201.127 |       4 |    4 |          3 |
| 2026-11-05 17:51:00 | 203.221.106.63  |      22 |   23 |         19 |
| 2026-11-05 17:50:00 | 203.221.106.63  |      22 |   23 |         19 |
| 2026-11-05 17:49:00 | 203.221.106.63  |      22 |   23 |         19 |
| 2026-11-05 17:48:00 | 203.221.106.63  |      22 |   23 |         19 |
| 2026-11-05 17:47:00 | 203.221.106.63  |      22 |   23 |         19 |
| 2026-11-05 17:46:00 | 203.221.106.63  |      22 |   23 |         19 |
| 2026-11-05 17:45:00 | 203.221.106.63  |      22 |   23 |         19 |
| 2026-11-05 17:44:00 | 203.221.106.63  |      22 |   23 |         19 |
| 2026-11-05 17:43:00 | 203.221.106.63  |      22 |   23 |         19 |
| 2026-11-05 17:40:00 | 147.135.16.215  |      17 |   17 |         14 |
| 2026-11-05 17:39:00 | 147.135.16.215  |      17 |   17 |         14 |
| 2026-11-05 17:38:00 | 147.135.16.215  |      17 |   17 |         14 |
| 2026-11-05 17:37:00 | 147.135.16.215  |      17 |   17 |         14 |
| 2026-11-05 17:36:00 | 147.135.16.215  |      17 |   17 |         14 |
| 2026-11-05 17:35:00 | 147.135.16.215  |      17 |   17 |         14 |
| 2026-11-05 17:34:00 | 147.135.16.215  |      17 |   17 |         14 |
| 2026-11-05 17:33:00 | 147.135.16.215  |      17 |   17 |         14 |
| 2026-11-05 17:32:00 | 147.135.16.215  |      17 |   17 |         14 |
| 2026-11-05 17:31:00 | 147.135.16.215  |      17 |   17 |         14 |
| 2026-11-05 00:59:00 | 115.4.176.81    |      15 |   15 |         10 |
| 2026-11-05 00:58:00 | 115.4.176.81    |      15 |   15 |         10 |
| 2026-11-05 00:57:00 | 115.4.176.81    |      15 |   15 |         10 |
| 2026-11-05 00:56:00 | 115.4.176.81    |      15 |   15 |         10 |
| 2026-11-05 00:55:00 | 115.4.176.81    |      15 |   15 |         10 |
| 2026-11-05 00:52:00 | 111.145.248.103 |      33 |   33 |         19 |
| 2026-11-05 00:51:00 | 111.145.248.103 |      33 |   33 |         19 |
| 2026-11-05 00:50:00 | 111.145.248.103 |      33 |   33 |         19 |
| 2026-11-05 00:49:00 | 111.145.248.103 |      33 |   33 |         19 |
| 2026-11-05 00:49:00 | 48.39.94.203    |      26 |   27 |         22 |
| 2026-11-05 00:48:00 | 111.145.248.103 |      33 |   33 |         19 |
| 2026-11-05 00:48:00 | 48.39.94.203    |      26 |   27 |         22 |
| 2026-11-05 00:47:00 | 111.145.248.103 |      33 |   33 |         19 |
| 2026-11-05 00:47:00 | 48.39.94.203    |      26 |   27 |         22 |
| 2026-11-05 00:46:00 | 111.145.248.103 |      33 |   33 |         19 |
| 2026-11-05 00:46:00 | 48.39.94.203    |      26 |   27 |         22 |
| 2026-11-05 00:45:00 | 111.145.248.103 |      33 |   33 |         19 |
| 2026-11-05 00:45:00 | 48.39.94.203    |      26 |   27 |         22 |
| 2026-11-05 00:44:00 | 111.145.248.103 |      33 |   33 |         19 |
| 2026-11-05 00:44:00 | 48.39.94.203    |      26 |   27 |         22 |
+---------------------+-----------------+---------+------+------------+
```

ip_alerts : les 50 dernières lignes

3.C Fraude visible ? (suite)

En tête, l'IP **144.161.201.127** (run actuel). Viennent ensuite deux IP de l'essai précédent : **203.221.106.63** (22 cartes, 23 transactions, 19 refus ; une carte a servi deux fois) et **147.135.16.215** (17 cartes, 17 transactions, 14 refus). Puis les lignes de la veille, qui comptent cinq autres IP du même type (15 à 33 cartes, 58 à 86 % de refus).

Deuxième requête de l'énoncé, sur l'IP qui a le plus de cartes :

3.C Fraude visible ? (suite)

```
(tp4) mfpadonou@cloudshell:~/tp4 (project-64f95bfc-58b9-4be1-aa5)$ bq query --u
se_legacy_sql=false \
  'SELECT window_start, window_end, n_cards, n_tx, n_declined
   FROM shopflow_rt.ip_alerts WHERE ip = "144.161.201.127" ORDER BY window_star
t'
+---------------------+---------------------+---------+------+------------+
|    window_start     |     window_end      | n_cards | n_tx | n_declined |
+---------------------+---------------------+---------+------+------------+
| 2026-11-05 18:29:00 | 2026-11-05 18:39:00 |       4 |    4 |          3 |
| 2026-11-05 18:30:00 | 2026-11-05 18:40:00 |      15 |   15 |         13 |
| 2026-11-05 18:31:00 | 2026-11-05 18:41:00 |      15 |   15 |         13 |
| 2026-11-05 18:32:00 | 2026-11-05 18:42:00 |      15 |   15 |         13 |
| 2026-11-05 18:33:00 | 2026-11-05 18:43:00 |      15 |   15 |         13 |
| 2026-11-05 18:34:00 | 2026-11-05 18:44:00 |      15 |   15 |         13 |
| 2026-11-05 18:35:00 | 2026-11-05 18:45:00 |      15 |   15 |         13 |
| 2026-11-05 18:36:00 | 2026-11-05 18:46:00 |      15 |   15 |         13 |
| 2026-11-05 18:37:00 | 2026-11-05 18:47:00 |      15 |   15 |         13 |
| 2026-11-05 18:38:00 | 2026-11-05 18:48:00 |      15 |   15 |         13 |
| 2026-11-05 18:39:00 | 2026-11-05 18:49:00 |      11 |   11 |         10 |
+---------------------+---------------------+---------+------+------------+
(tp4) mfpadonou@cloudshell:~/tp4 (project-64f95bfc-58b9-4be1-aa5)$
```

Toutes les lignes de l'IP 144.161.201.127

3.C Fraude visible ? (suite)

Relecture des messages dans Kafka (script `attaque.py`, qui lit les 1 500 derniers messages de chaque partition et garde ceux de l'IP) :

3.C Fraude visible ? (suite)

```
(tp4) mfpadonou@cloudshell:~/tp4 (project-64f95bfc-58b9-4be1-aa5)$ python attaq
ue.py 144.161.201.127
event_ts       partition card_id status
18:38:55.947 5 438931 authorized
18:38:56.564 3 161741 declined
18:38:56.572 1 422111 declined
18:38:57.522 3 378771 declined
18:39:00.639 0 122131 authorized
18:39:01.289 5 285901 declined
18:39:02.239 3 165661 declined
18:39:02.314 2 284051 declined
18:39:03.805 0 26431 declined
18:39:03.939 0 137601 declined
18:39:04.714 4 194231 declined
18:39:04.855 2 72311 declined
18:39:09.239 1 221961 declined
18:39:09.355 5 150371 declined
18:39:10.022 3 187601 declined
15 paiements, 15 cartes, 13 refus
(tp4) mfpadonou@cloudshell:~/tp4 (project-64f95bfc-58b9-4be1-aa5)$
```

15 paiements entre 18:38:55.9 et 18:39:10.0, soit une rafale de 14 s, avec **15** `card_id` **différents**, répartis sur **les 6 partitions**, et 13 `declined` (87 %).

3.C Fraude visible ? (suite)

Le tableau de bord de la partie 4 l'affiche aussi dans « IP suspectes » (capture en partie 4).

Q10. Oui : une IP qui essaie **beaucoup de cartes différentes en quelques secondes**, avec **80 à 90 % de refus** (13 sur 15 ici, 14 sur 17 lors du premier essai). C'est le scénario **test de cartes volées** (*card testing*) du fil rouge : l'attaquant vérifie quelles cartes d'une liste volée fonctionnent, et les refus dominent.

La même attaque apparaît dans **plusieurs lignes**. La fenêtre fait 10 minutes et glisse d'une minute, donc chaque paiement appartient à **10 fenêtres**. Ici, la rafale chevauche la minute 18:39 (4 paiements avant, 11 après) :

- fenêtre 18:29-18:39 : 4 cartes, 3 refus (seulement le début de la rafale) ;
- fenêtres 18:30-18:40 à 18:38-18:48 : 15 cartes, 13 refus (toute la rafale) ;
- fenêtre 18:39-18:49 : 11 cartes, 10 refus (seulement la fin).

3.C Fraude visible ? (suite)

Soit **11 lignes pour une seule attaque**, et 10 lignes identiques lors du premier essai, dont la rafale tenait dans une seule minute. Pour alerter, il faut dédupliquer par IP (par exemple ne garder que la première fenêtre au-dessus du seuil), sinon l'équipe fraude reçoit 10 ou 11 alertes pour une attaque.

À faire valoir en séance :

3.C Fraude visible ? (suite)

- **Délai de détection.** La première ligne d'alerte est chargée dans BigQuery à **18:42:05-18:42:09**, soit **3 min 13 s après le premier paiement frauduleux** (18:38:55.9). Lors du premier essai, on avait 3 min 49 s (attaque à 17:40:20, alerte à 17:44:09). Le délai varie de ~3 min 10 s à ~4 min 10 s selon la position de l'attaque dans la minute. **L'objectif de moins de 3 minutes du contexte n'est pas atteint** avec cette configuration (watermark de 2 min, déclencheur de 30 s ; voir Q7). Leviers : un watermark plus court, un déclencheur plus fréquent, ou le mode *update* vers une destination capable de mettre à jour une ligne.
- **Partitionnement.** Les cartes sont réparties sur les 6 partitions, car la clé Kafka est `card_id`. Compter par IP impose donc un **shuffle** (`groupBy("ip")`). C'est le pendant de Q1 : la clé de partitionnement sert les traitements par carte, pas ceux par IP.
- **Le taux de refus global ne voit rien.** Pendant l'attaque, le tableau de bord affichait 2,9 % de refus sur 5 minutes : 13 refus noyés dans environ 1 000 paiements. Seul le suivi par IP révèle le test de cartes.

3.C Fraude visible ? (suite)

- **Anomalie dans les lignes de la veille** : l'IP `10.29.150.57` est une adresse privée (RFC 1918), impossible pour un client vu d'Internet. Elle vient du générateur, et peut servir de question de qualité de données.

Partie 4 : tableau de bord

`dashboard.py` est identique à l'énoncé (md5 `f79f348a…`, 43 lignes). Lancement avec la commande de l'énoncé :

Partie 4 : tableau de bord (suite)

```
(tp4) mfpadonou@cloudshell:~/tp4 (project-64f95bfc-58b9-4be1-aa5)$ md5sum dashb
oard.py
streamlit run dashboard.py --server.port 8080 --server.enableCORS false --serve
r.enableXsrfProtection false
f79f348a6acf29fdca5e4a1a14a1b293  dashboard.py

Collecting usage statistics. To deactivate, set browser.gatherUsageStats to fal
se.

2026-11-05 18:36:14.549 Uvicorn server started on :::8080

  You can now view your Streamlit app in your browser.

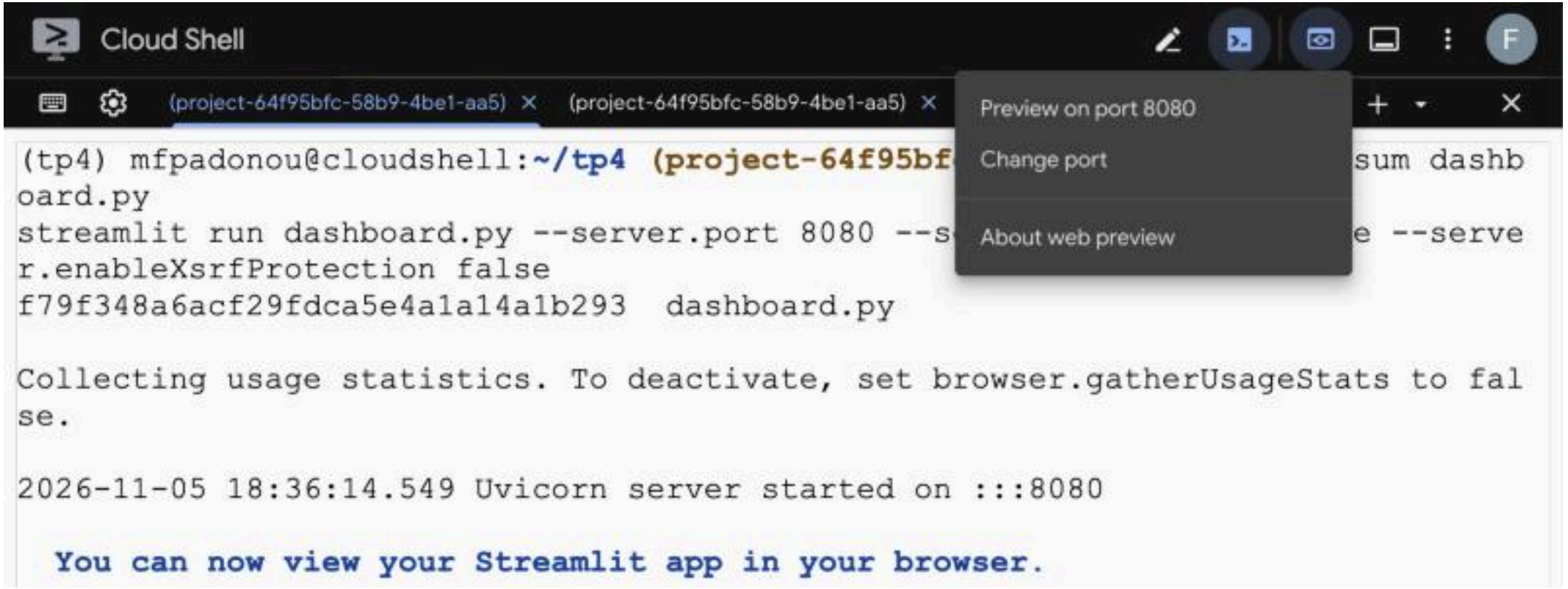
  Local URL: http://localhost:8080
  Network URL: http://10.88.0.3:8080
  External URL: http://34.79.240.96:8080
```

streamlit run

Partie 4 : tableau de bord (suite)

Le menu **Aperçu sur le Web** de Cloud Shell propose **Preview on port 8080** :

Partie 4 : tableau de bord (suite)



Aperçu sur le Web

Partie 4 : tableau de bord (suite)

Le tableau de bord s'ouvre à une adresse de la forme `https://8080-cs-….cloudshell.dev/` :

Partie 4 : tableau de bord (suite)

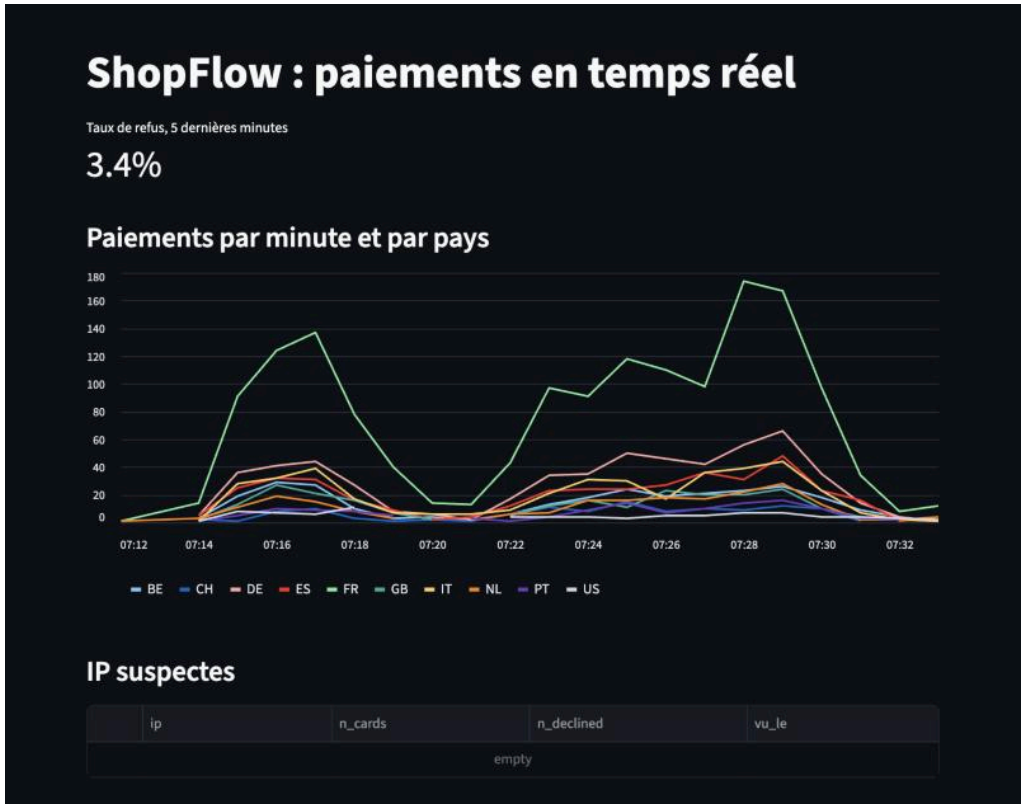


Tableau de bord Streamlit

Partie 4 : tableau de bord (suite)

Observations :

- **Le rafraîchissement automatique fonctionne** : entre deux captures à 15 s d'intervalle, le taux de refus est passé de 3,0 % à 3,4 % et une minute s'est ajoutée à la courbe.
- **Taux de refus de 3 à 3,4 %** sur 5 minutes, hors attaque. Un test de cartes (80 % de refus sur une IP) le ferait monter nettement.
- La courbe par pays montre la même respiration du trafic que les courbes Spark. FR domine, avec jusqu'à 175 paiements par minute.
- **L'axe des temps s'affiche en heure locale du navigateur, au format 12 h sans AM/PM** : `07:12` correspond à 19:12 à Paris, soit 18:12 UTC. Les étudiants qui comparent avec `bq query` (en UTC) peuvent croire à une erreur.
- **« IP suspectes » est vide** tant qu'aucune IP n'a franchi 3 cartes dans les 30 dernières minutes (filtre `window_end > now − 30 min`).

Partie 4 : tableau de bord (suite)

Après l'attaque de 18:39 (voir 3.C), le tableau affiche l'IP, avec le maximum de cartes et de refus sur ses fenêtres :

Partie 4 : tableau de bord (suite)

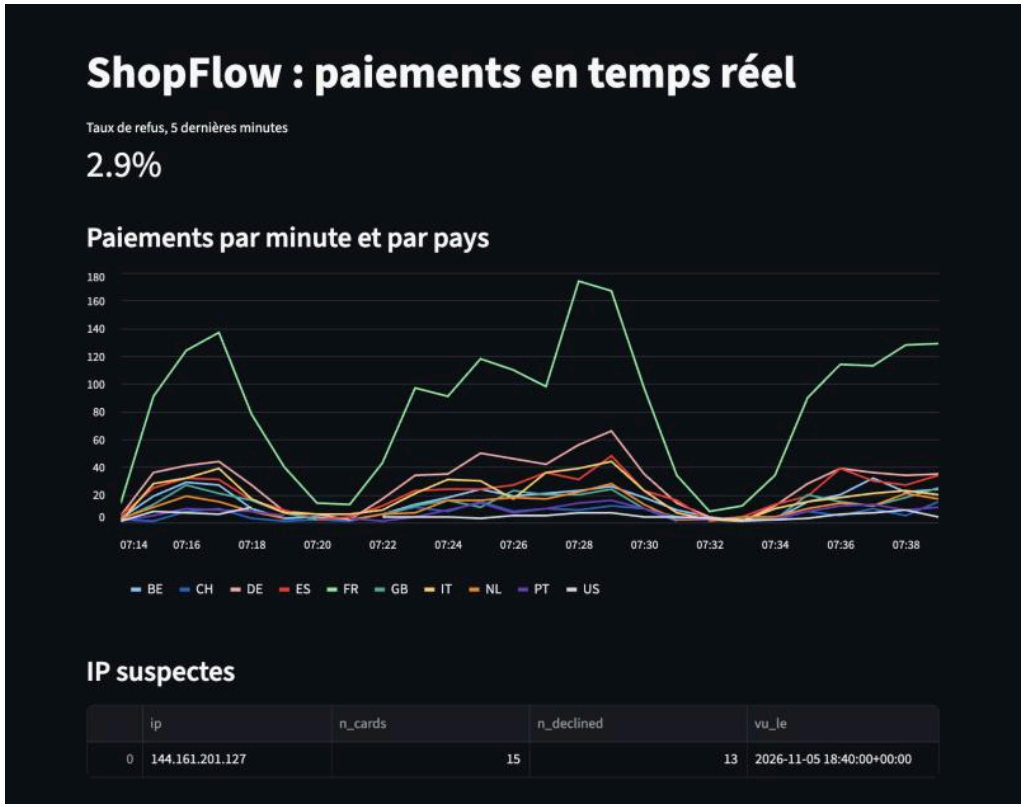


Tableau de bord pendant l'attaque

Partie 4 : tableau de bord (suite)

Le taux de refus global reste à **2,9 %** : l'attaque ne se voit que dans la liste des IP.

Coût des requêtes du tableau de bord

Lu dans `INFORMATION_SCHEMA.JOBS_BY_PROJECT` :

Coût des requêtes du tableau de bord (suite)

```
(tp4) mfpadonou@cloudshell:~/tp4 (project-64f95bfc-58b9-4be1-aa5)$ bq query --u
se_legacy_sql=false \
  'SELECT IF(query LIKE "%ip_alerts%", "ip_alerts", "payments_by_minute") AS re
quete,
          COUNT(*) AS n, ROUND(AVG(total_bytes_processed)) AS octets_lus,
          ROUND(AVG(total_bytes_billed) / 1e6, 1) AS mo_factures
   FROM `region-eu`.INFORMATION_SCHEMA.JOBS_BY_PROJECT
   WHERE job_type = "QUERY" AND creation_time > TIMESTAMP "2026-11-05 18:36:00"

     AND query LIKE "%INTERVAL 30 MINUTE%" AND query NOT LIKE "%JOBS_BY_PROJECT
%"
   GROUP BY requete'
+--------------------+---+-----------+-------------+
|      requete       | n | octets_lus | mo_factures |
+--------------------+---+-----------+-------------+
| payments_by_minute | 3 |    32527.0 |        10.5 |
| ip_alerts          | 3 |     2900.0 |        10.5 |
+--------------------+---+-----------+-------------+
(tp4) mfpadonou@cloudshell:~/tp4 (project-64f95bfc-58b9-4be1-aa5)$
```

Octets lus et facturés par requête du tableau de bord

Coût des requêtes du tableau de bord (suite)

Chaque rafraîchissement lance 2 requêtes. Elles **lisent** 32 Ko (`payments_by_minute`) et 3 Ko (`ip_alerts`), mais BigQuery en **facture 10,5 Mo chacune** : c'est le minimum facturé par table en mode à la demande. Avec un rafraîchissement toutes les 15 s, cela donne 5 760 rafraîchissements par jour, 11 520 requêtes et **≈ 121 Go facturés par jour et par écran ouvert**, pour quelques Mo réellement utiles. Le palier gratuit de 1 To par mois serait épuisé en 8 à 9 jours par un seul écran (tarification BigQuery). C'est la réponse chiffrée à la Q11 du corrigé enseignant (question absente de l'énoncé actuel). Leviers : rafraîchir moins souvent, mettre en cache côté application (`st.cache_data(ttl=…)`), partitionner les tables par `window_start`, ou servir le tableau de bord depuis une base de service.

Le rafraîchissement tourne **par session de navigateur connectée** (`st.fragment(run_every=…)`) : le coût augmente avec le nombre d'écrans ouverts. Un onglet en arrière-plan rafraîchit moins souvent (3 exécutions en 2 min mesurées dans ce cas).

Recommandations pour l'énoncé et le corrigé enseignant

1. **3.A** : vérifier un cluster existant (`clusters describe … --format='yaml(config.gceClusterConfig.internalIpOnly,config.lifecycleConfig)'`) et le supprimer s'il vient du TP 3. Prévenir que `bq mk` répond « already exists » quand le dataset existe.
2. **3.B** : supprimer `$BUCKET/tp4/checkpoints/` avant toute nouvelle soumission.
3. **Interface Spark** :

- adapter le chemin (« Apache Spark géré », section *Interfaces Web de clusters*, YARN puis **RUNNING**) ;
- corriger le nom de la métrique (*Late Rows Dropped By Watermark*) ;
- prévenir que les graphiques de la page de détail restent vides via la gateway. Pour Q9, faire relever les colonnes *Avg Input/sec* et *Latest Batch* de la liste, et donner le nombre de lignes ignorées autrement : JSON `Streaming query made progress` dans les journaux du driver (`numRowsDroppedByWatermark`), ou valeurs fournies par l'enseignant.

Recommandations pour l'énoncé et le corrigé enseignant (suite)

4. **Q7** : remplacer « 150 à 240 s » par « ~190 à ~245 s » et « 10 h 03 m 30 s à 10 h 04 » par « vers 10 h 04 m 07 s ».
5. **Cloud Shell** :

- chaque nouvel onglet crée sa propre configuration gcloud (`cloudshell-NNNNN`), parfois **sans compte actif** (`You do not currently have an active account selected`). Correctif : `gcloud config set account <adresse>` ;
- si l'onglet du navigateur reste longtemps en arrière-plan, `bq` peut échouer avec `service account info is missing 'email' field` : **recharger la page Cloud Shell** rétablit les identifiants ;
- à ajouter à la section « Pièges fréquents ».

6. **Partie 4** : signaler l'affichage des heures (local, 12 h). Remettre une question sur le coût (Q11 du corrigé), avec les chiffres mesurés ci-dessus.

Recommandations pour l'énoncé et le corrigé enseignant (suite)

7. **Quota BigQuery** : le connecteur lance un job LOAD par table toutes les 30 s, soit 2 880 par jour. BigQuery limite à **1 500 jobs de chargement par table et par jour** (quotas). **Un job laissé tournant plus de ~12 h 30 atteindra ce quota et échouera.** En TP, `--delete-max-age=4h` protège contre ce risque. En production : déclencheur plus long, ou `writeMethod=direct` (API Storage Write).

État des ressources laissées actives

À la demande de l'utilisateur, **rien n'a été arrêté ni supprimé** à la fin du déroulé :

Ressource	État	Remarque
Cluster `shopflow-spark` (1 maître + 2 workers n2-standard-4)	RUNNING, **sans arrêt automatique**	Facturé en continu jusqu'à sa suppression manuelle
Job de streaming `dbf541f391b94f91a102ee9a2d864c95`	RUNNING depuis 18:14 UTC	À 2 880 chargements par table et par jour, il dépassera le quota de 1 500 environ 12 h 30 après son démarrage (vers 06:45 UTC le 6/11, selon la façon dont BigQuery compte la journée) et échouera alors (recommandation 7)
Tables `shopflow_rt.*`, checkpoints `$BUCKET/tp4/checkpoints/`	Alimentées par le job	—
Streamlit dans Cloud Shell (port 8080)	**Arrêté** à 18:56 UTC (`pkill -f "streamlit run"`)	Dans tous les cas, il s'arrête avec la session Cloud Shell : ce n'est pas un déploiement durable

Pour tout arrêter, utiliser la fin de l'énoncé :

```
gcloud dataproc jobs kill dbf541f391b94f91a102ee9a2d864c95 --region=europe-west1
gcloud dataproc clusters delete shopflow-spark --region=europe-west1
pkill -f streamlit
```